Programming Language 1
Python

Dr. Samera Obaid Barracood

VARIABLES AND DATA TYPES IN PYTHON

Variables

- Reserved memory locations to store values.
- It means that when you create a variable, you reserve some space in the memory.

```
counter = 100          # An integer assignment
miles = 1000.0         # A floating point
name = "John"          # A string
print (counter)
print (miles)
print (name)
```

Variables

- Variables are used to store data
- These data are stored in main memory when you run the program
- Different data types in python
  - Numbers
  - Strings
  - Lists
  - Tuple
  - Dictionary

Multiple Assignment

- Python allows you to assign a single value to several variables simultaneously.

  a = b = c = 1

- You can also assign multiple objects to multiple variables

  a, b, c = 1, 2, "john”

RULES FOR NAMING VARIABLES

RULE 1 - Variable names can contain only letters, numbers, and underscores. They can start with a letter or an underscore, but not with a number.

```
>>> message = "my name is shivam"
>>> print(message)
my name is shivam
>>> _message = "my name is shivam"
>>> print(_message)
my name is shivam
```

SUPPOSE YOU WANT TO WRITE TWO DIFFERENT MESSAGES ?

```
>>> 1_meesage = "my name is shivam"
```

```
>>> 1_message = "my name is shivam"
  File "<stdin>", line 1
    1_message = "my name is shivam"
      ^
SyntaxError: invalid token
```


>>> message_1 = "my name is shivam"
>>> print(message_1)
my name is shivam
>>> message_2 = "welcome to my python
tutorial"
>>> print(message_2)
welcome to my python tutorial

RULE 2 - Spaces are not allowed in variable names, but underscores can be used to separate words in variable names.

```
>>> message 1 = "my name is shivam"
```

```
>>> message 1 = "my name is shivam"
  File "<stdin>", line 1
    message 1 = "my name is shivam"
              ^
SyntaxError: invalid syntax
```

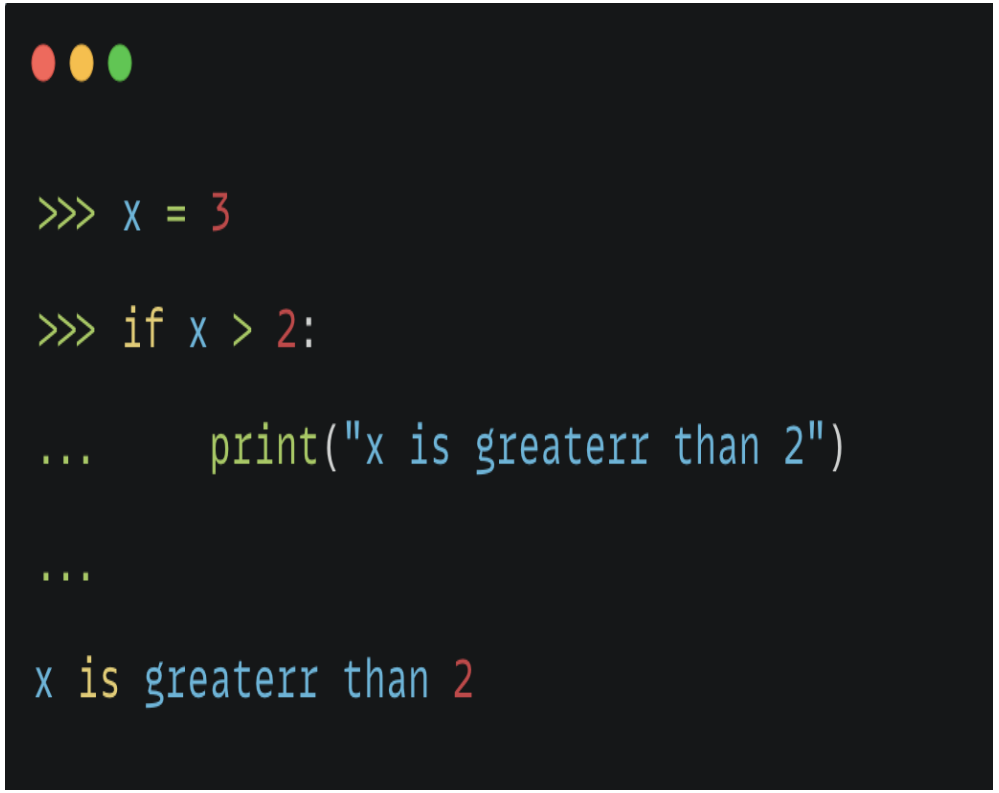
RULE 3 - Avoid using Python keywords and function names as variable names; that is, do not use words that Python has reserved for a particular programmatic purpose, such as the word print.

```
>>> if = "my name is shivam"
```

```
>>> if = "my name is shivam"
  File "<stdin>", line 1
    if = "my name is shivam"
       ^
SyntaxError: invalid syntax
```

WHAT ARE KEYWORDS IN PYTHON?

- These are reserved words in python which are used to define the syntax of python language.
- In English, “is”, “the”, “you” etc. are reserved words which defines the grammar.
- Keywords have different meaning for Python language.
- Do not use them in your variables or function names.



Keywords

Keywords are the reserved words that are already defined by the Python for specific uses

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	as	assert
async	def	del	elif	from
global	if	lambda	try	not
or	while	with	yield	nonlocal

```
>>> print = "my name is shivam"
```

```
>>> print = "my name is shivam"
>>> print(print)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object is not callable
```

BUILT-IN FUNCTIONS IN PYTHON

You won't get an error if you use one of the following readily available built-in functions as a variable name, but you'll override the behavior of that function:

abs()	delattr()	hash()	memoryview()	set()
all()	dict()	help()	min()	setattr()
any()	dir()	hex()	next()	slice()
ascii()	divmod()	id()	object()	sorted()
bin()	enumerate()	input()	oct()	staticmethod()
bool()	eval()	int()	open()	str()
breakpoint()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
compile()	globals()	map()	reversed()	__import__()
complex()	hasattr()	max()	round()	

SOME OTHER IMPORTANT POINTS TO NOTE

- Variable names should be short but descriptive.
  - name is better than writing n
  - message is better than writing m or my_name_message
- Be careful when using the lowercase letter *l* and the uppercase letter *O* because they could be confused with the numbers *1* and *0*.

```
>>> Office_city = "Mumbai"
>>> 0ffice_city = "Mumbai"
  File "<stdin>", line 1
    0ffice_city = "Mumbai"
                ^
SyntaxError: invalid syntax
```

```
>>> letter = "a"
>>> 1etter = "a"
  File "<stdin>", line 1
    1etter = "a"
          ^
SyntaxError: invalid syntax
```

It can take some practice to learn how to create good variable names, especially as your programs become more interesting and complicated. As you write more programs and start to read through other people’s code, you’ll get better at coming up with meaningful names.

AVOIDING NAME ERRORS WHEN USING VARIABLES

```
>>> message = "my name is shivam"
>>> print(mesage)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'mesage' is not defined
```

Operators

Python language supports the following types of operators-

- Arithmetic Operators
- Comparison (Relational) Operators
- Assignment Operators
- Logical Operators
- Bitwise Operators
- Membership Operators
- Identity Operators

Arithmetic Operators

Assume a=10 ,b =21, then … print(C)

Operator	Description	Example
+ Addition	Adds values on either side of the operator.	C=a + b C = 31
- Subtraction	Subtracts right hand operand from left hand operand.	C=a – b C = 31 = -11
* Multiplication	Multiplies values on either side of the operator	C=a * b C = 31 = 210
/ Division	Divides hand operand by right hand operand	C= b / a C = 31 =2.1
% Modulus	Divides left hand operand by right hand operand and returns remainder	C=b % a C = 31 = 1
** Exponent	Performs exponential (power) calculation on operators	C=a**4 C =10000
// Floor Division	- The division of operands where the result is the quotient in which the digits after the decimal point are removed.	C=b//2 C=10

Comparison Operators/ Relational operators

- These operators compare the values on either side of them and decide the relation among them.
- Assume a =10 , b=20, then

Operator<	Description	Example
==	If the values of two operands are equal, then the condition become true	if(a==b): Print('is not true')
!=	If values of two operands are not equal, then condition become true	if(a!=b) : Print('is true')
>	If the value of the left operand is greater than the value of the right operand, then condition become true	if(a>b) : Print(' is not true')
<	If the value of the left operand is less than the value of the right operand, then condition become true	if(a<b) : Print('is true')
>=	If the value of the left operand is greater than or equal to the value of the right operand, then condition become true	if(a>=b) : Print('is not true')
<=	If the value of the left operand is less than or equal to the value of the right operand, then condition become true	if(a<=b) : Print('is true')

Logical operators

Precedence of logical operator

- not –unary operator
- and
- or

Decreasing order

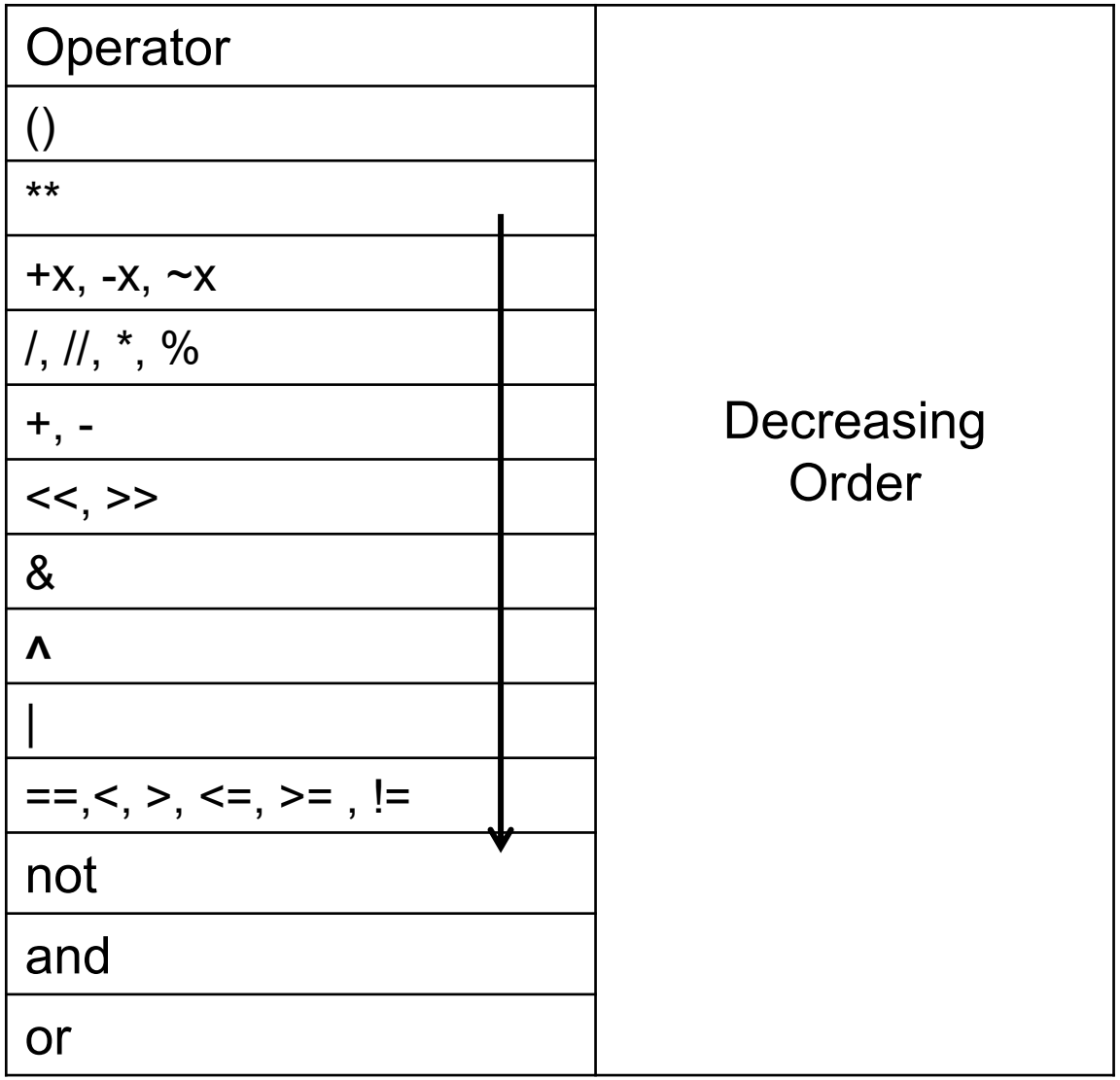
Example

```
>>> (10<5) and ((5/0)<10)
False
>>> (10>5) and ((5/0)<10)
ZeroDivisionError
```

Bitwise operators

- Bitwise AND – x & y
- Bitwise OR – x|y
- Bitwise Complement - ~ x
- Bitwise Exclusive OR -x^y
- Left Shift – x<<y
- Right Shift –x>>y

Precedence and Associativity of operators



Data Types

Python has five standard data types-

- Numbers
- String
- List
- Tuple
- Dictionary

FINDING THE DATA TYPE OF A VARIABLE

```
>>> message = "hello world"
>>> print(type(message))
<class 'str'>
>>> x = 2
>>> print(type(x))
<class 'int'>
```

- Use type() built-in function

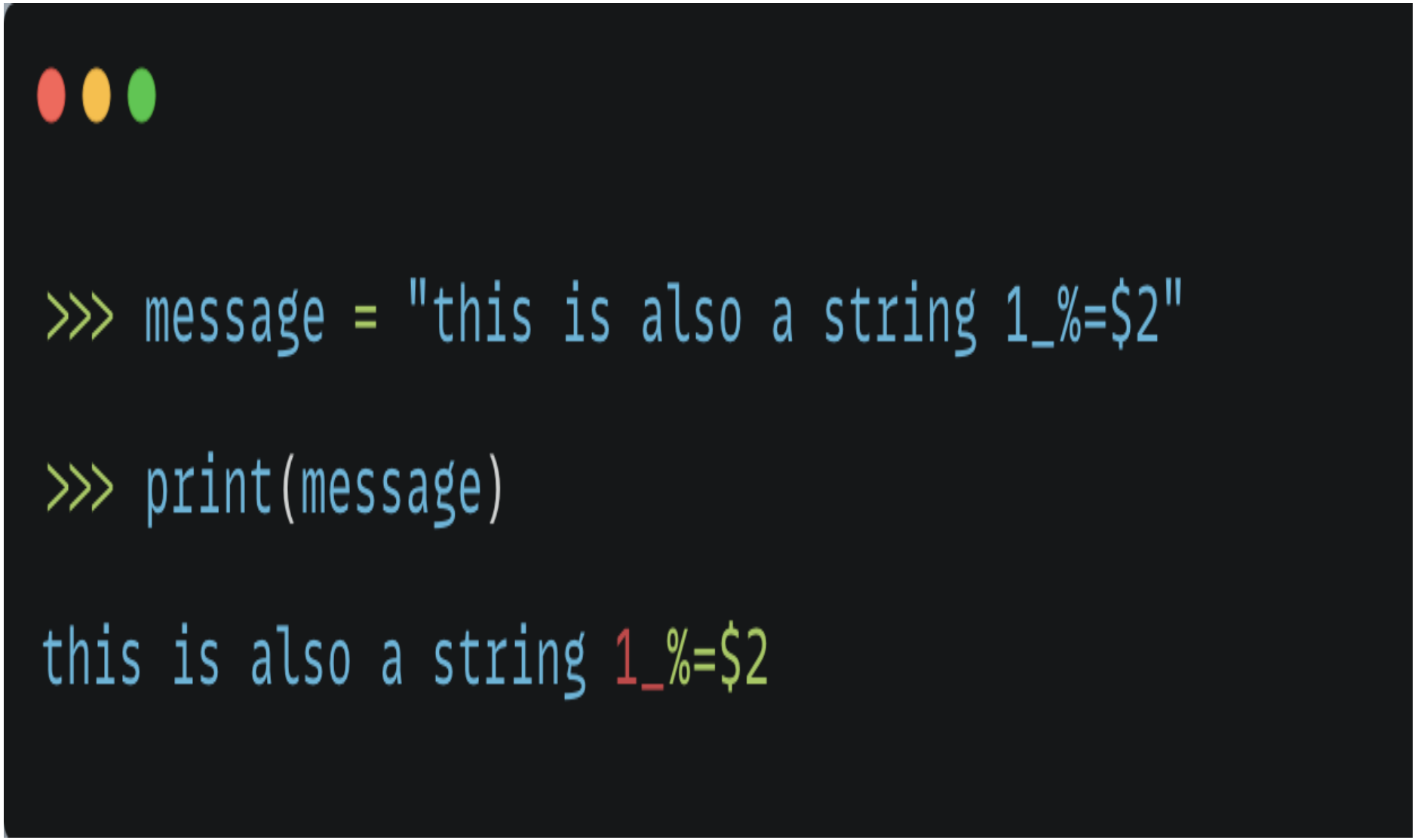
STRINGS DATA TYPE

WHAT ARE STRINGS ?

- A string is a series of characters
- Characters – alphabets, digits, special characters, white spaces
- Anything inside quotes is considered a string in Python
- You can use single or double quotes around your strings

```
>>> message = 'this is a string'
>>> print(message)
this is a string
>>> print(type(message))
<class 'str'>
```

```
>>> message = "this is a string"
>>> print(message)
this is a string
>>> print(type(message))
<class 'str'>
```


>>> message = "this is also a string 1_%=$2"
>>> print(message)
this is also a string 1_%=$2

SINGLE QUOTE VS DOUBLE QUOTE ?

- You can use any of these
- Be consistent
- Exceptions – print these text on screen:
  - The language 'Python' is named after Monty Python, not the snake.
  - I told my friend, "Python is my favorite language!"

```
>>> message = "The language 'Python' is named after Monty Python, not the snake."
>>> print(message)
The language 'Python' is named after Monty Python, not the snake.
>>> message = 'I told my friend, "Python is my favorite language!"'
>>> print(message)
I told my friend, "Python is my favorite language!"
```

ASSIGNMENT: Try to find out if it is possible to print first message by using single quotes and second by using double quotes and how ?

BUILT-IN FUNCTIONS ASSOCIATED WITH STRINGS

UPPER/LOWER FUNCTION

```
>>> message = "this is a string"
>>> print(message.upper())
THIS IS A STRING
```

```
>>> message = "THIS IS A STRING"
>>> print(message.lower())
this is a string
```

```
>>> message = "THIS IS a string"
>>> print(message.lower())
this is a string
```

- print() vs message.lower()
- These built-in functions are specific to strings.
- print() is specific to multiple data types
- The dot(.) operator acts on the variable and converts it into upper/lower cases
- The original data doesn’t change

```
>>> message = "this is a string"
>>> print(message.upper())
THIS IS A STRING
>>> print(message)
this is a string
```

```
>>> message = "this is a string"
>>> message = message.upper()
>>> print(message)
THIS IS A STRING
```

TITLE FUNCTION

Converts the first character of each word into upper case

```
>>> name = "shivam mitra"
>>> print(name.title())
Shivam Mitra
```

```
>>> name = "shivam mitra"
>>> name = name.title()
>>> print(name)
Shivam Mitra
```

LEN FUNCTION

- Outputs the length of the string
- A single character is of length 1
- Characters can be alphabets or digits or punctuation or white spaces

STRIP FUNCTIONS – strip(), lstrip(), rstrip()

Remove the extra whitespaces from a string

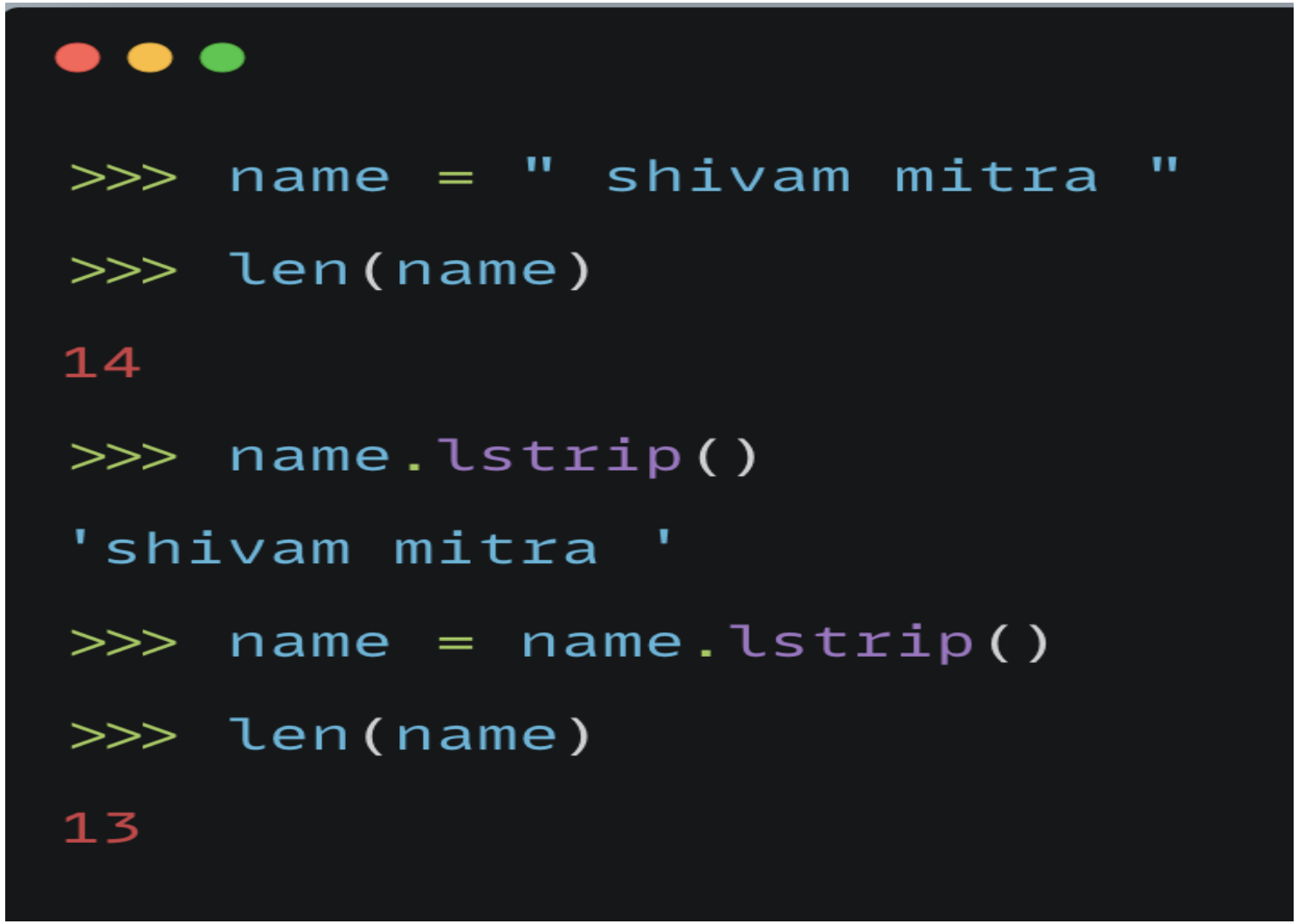
```
>>> string_1 = "python"
>>> string_2 = "python "
```

```
>>> string = "a"
>>> print(len(string))
1
>>> string = "abc1_"
>>> print(len(string))
5
>>> string = "abc1_ "
>>> print(len(string))
6
```

rstrip() – remove whitespace at right end

```
>>> name = " shivam mitra "
>>> len(name)
14
>>> name.rstrip()
' shivam mitra'
>>> name = name.rstrip()
>>> len(name)
13
```

lstrip() – remove whitespace from left end

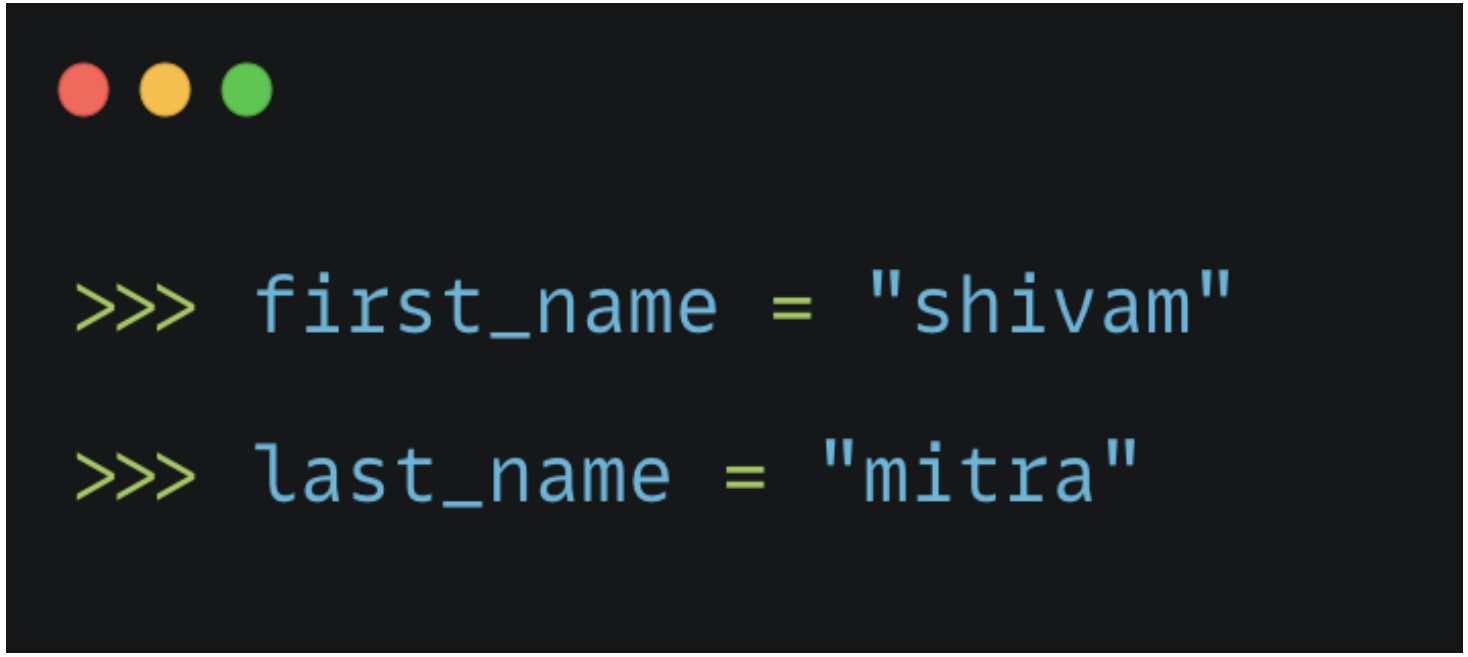


strip() – remove spaces from both ends

```
>>> name = " shivam mitra "
>>> len(name)
14
>>> name.strip()
'shivam mitra'
>>> name = name.strip()
>>> len(name)
12
```

USING VARIABLES INSIDE A STRING

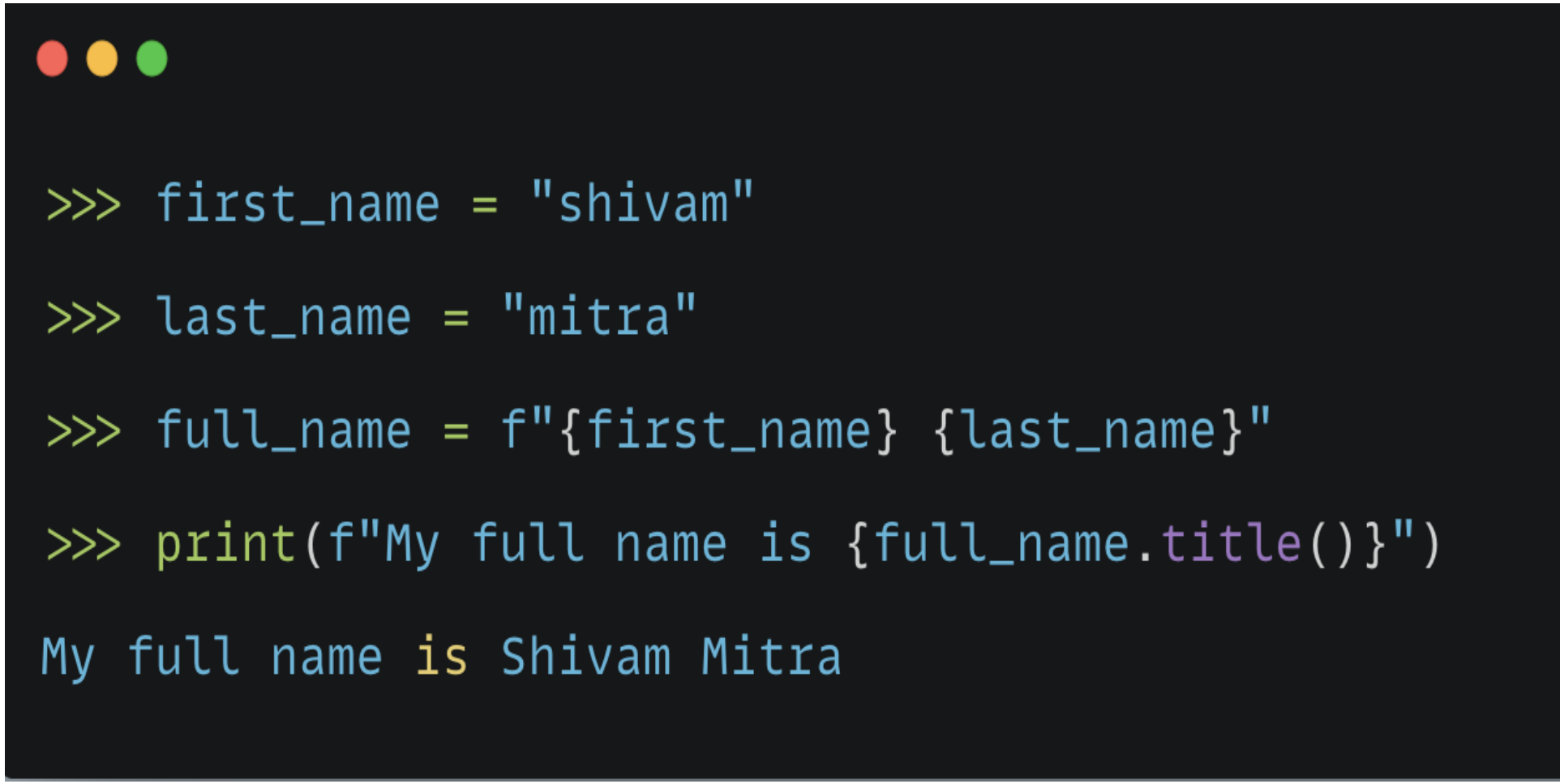
COMBINE THESE TWO TO PRODUCE FULL NAME

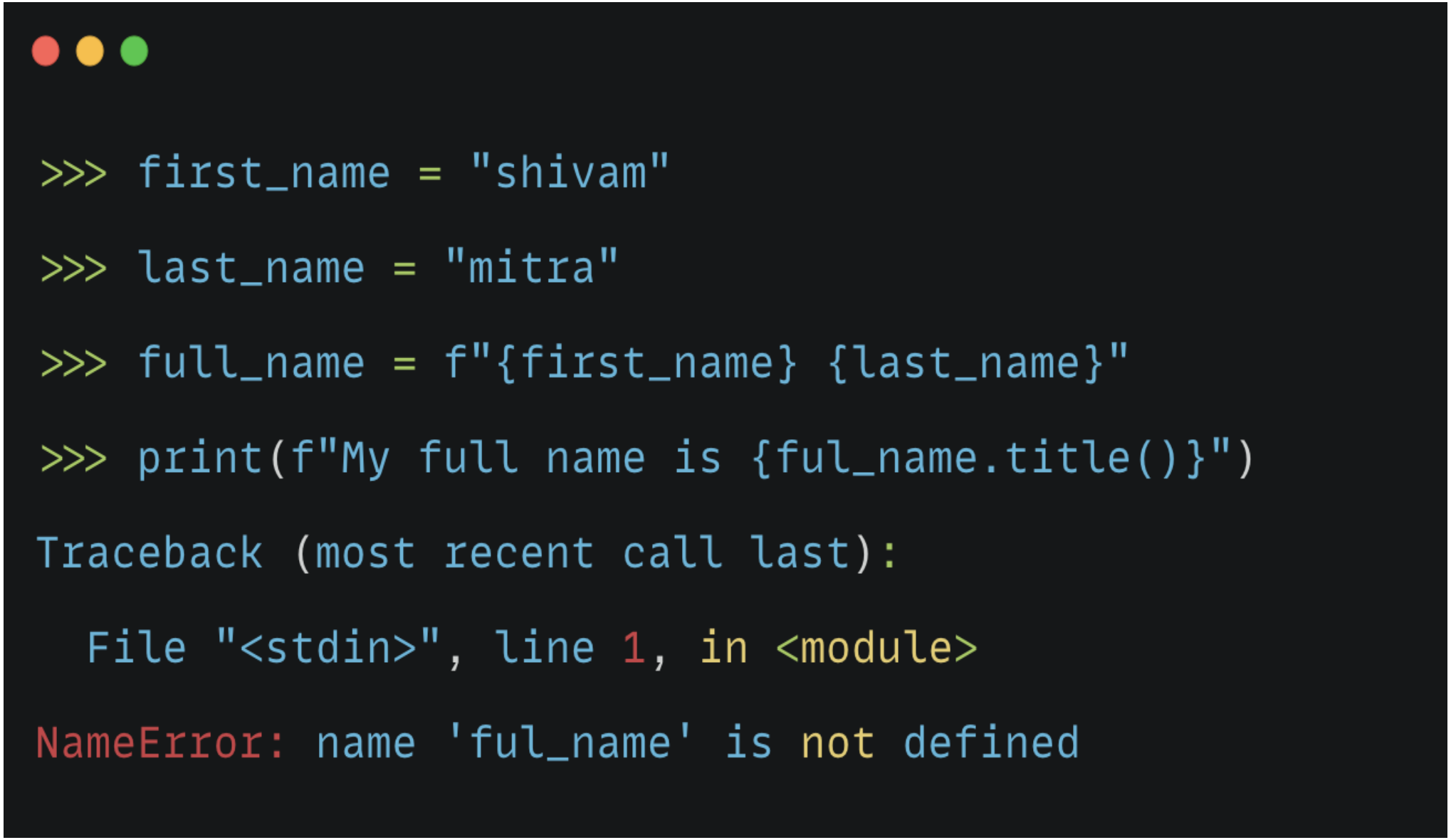


Use f-strings

- Feature is available from python 3.6
- Start string with f – f stands for format
- Put braces around variable name

```
>>> first_name = "shivam"
>>> last_name = "mitra"
>>> full_name = f"my full name is {first_name} {last_name}"
>>> print(full_name)
my full name is shivam mitra
```


>>> first_name = "shivam"
>>> last_name = "mitra"
>>> full_name = f"{first_name} {last_name}"
>>> print(f"My full name is {full_name.title()}")
My full name is Shivam Mitra


>>> first_name = "shivam"
>>> last_name = "mitra"
>>> full_name = f"{first_name} {last_name}"
>>> print(f"My full name is {ful_name.title()}")
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
NameError: name 'ful_name' is not defined

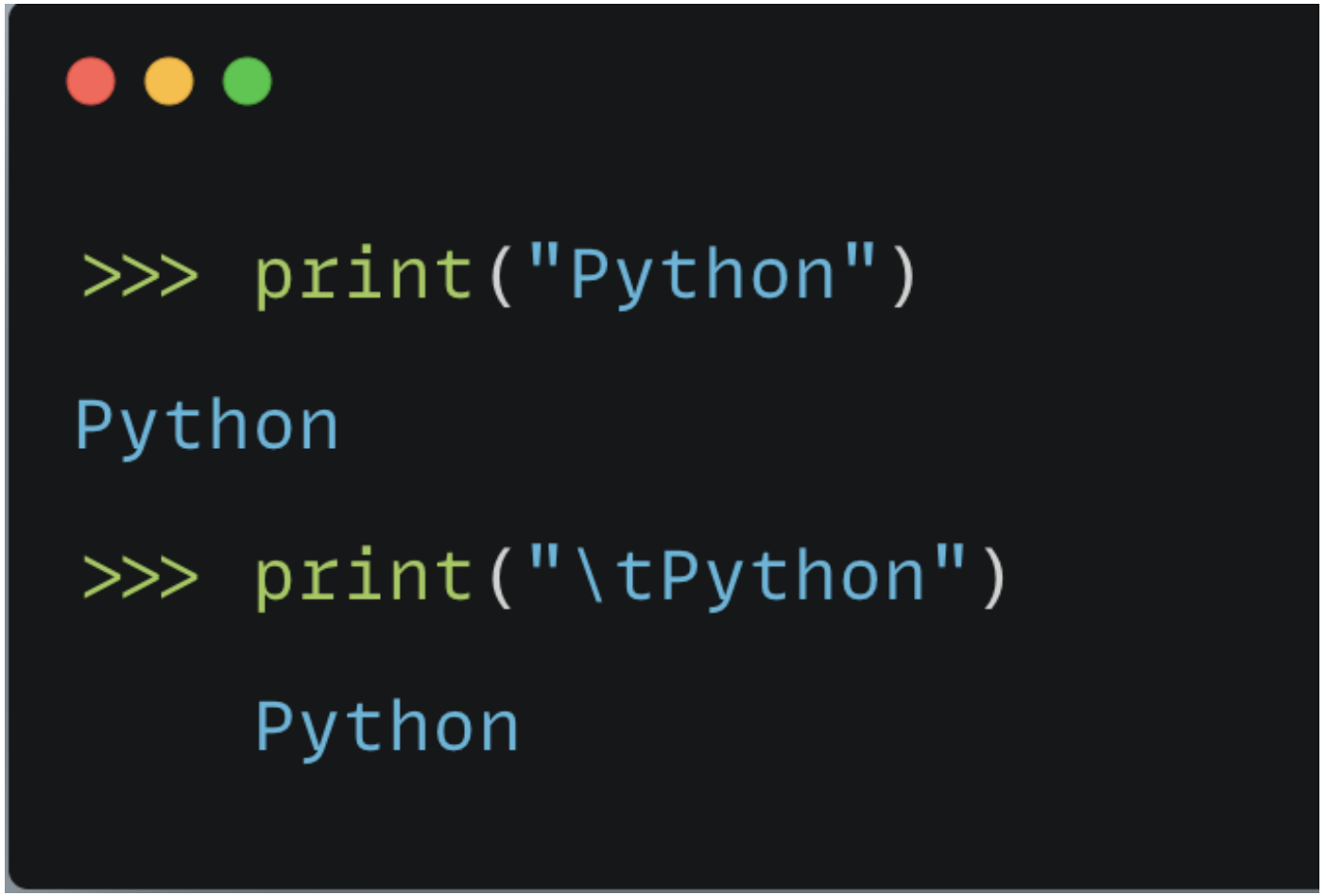
CONCAT MULTIPLE STRINGS

```
>>> first_name = "shivam"
>>> last_name = "mitra"
>>> full_name = first_name + " " + last_name
>>> print(full_name)
shivam mitra
```

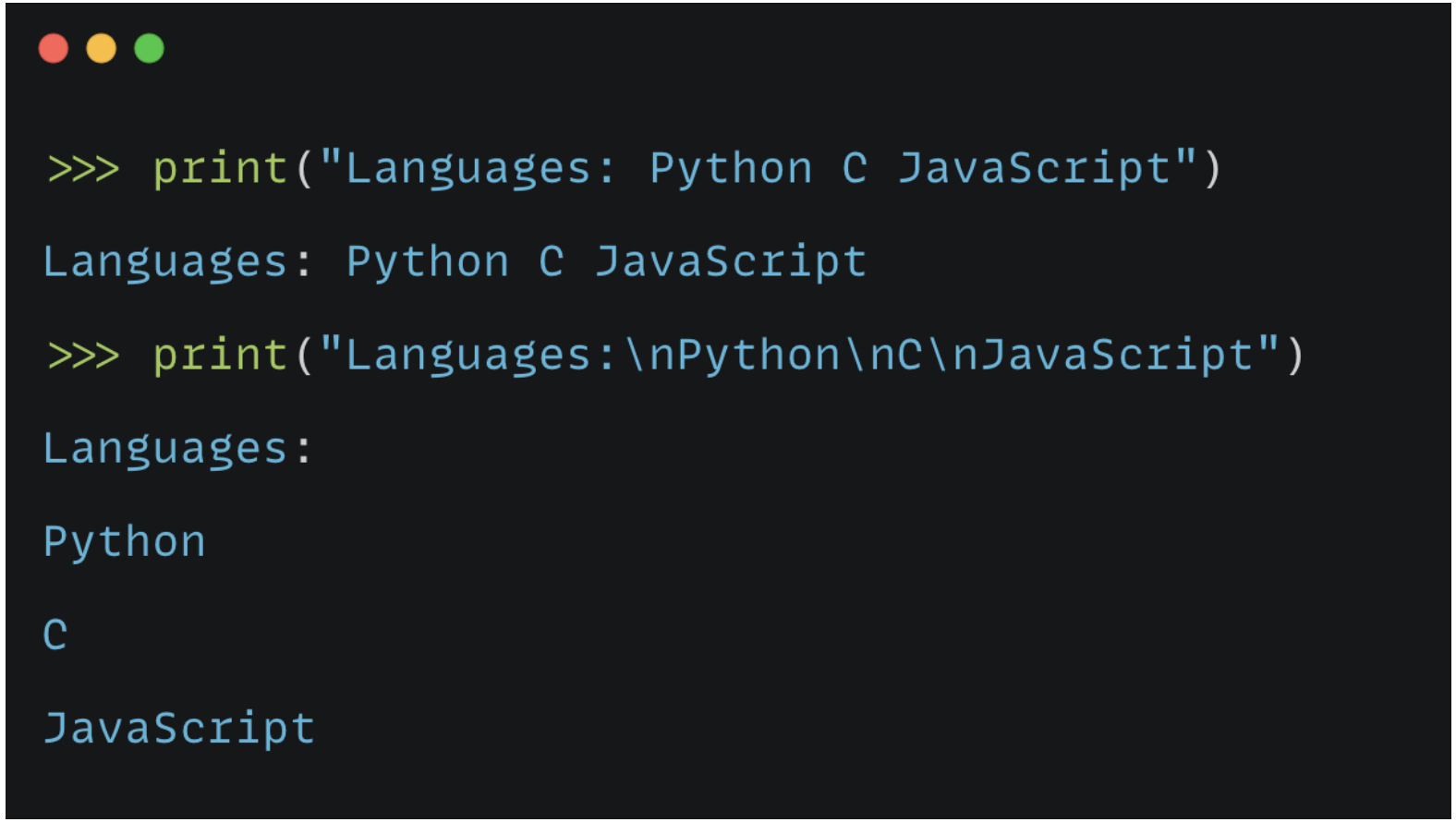
Adding Whitespace to Strings with Tabs or Newlines

- In programming, *whitespace* refers to any nonprinting character, such as spaces, tabs, and end-of-line symbols.
- You can use whitespace to organize your output so it's easier for users to read.

Add a tab to your text



Add newline to your text



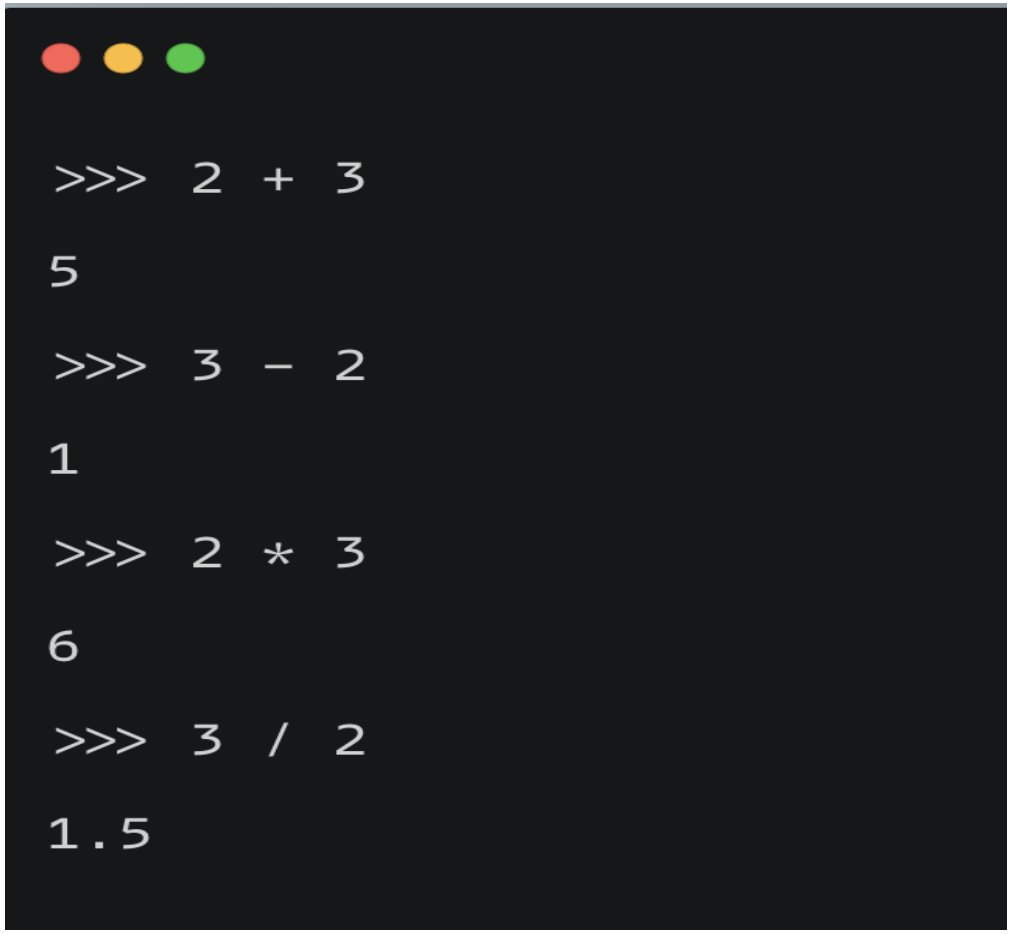
NUMBERS DATA TYPE

What are numbers ?

- Integers = 1, 2, 3
- Floats = 1.2, 2.55
- These data are used very frequently in a program.

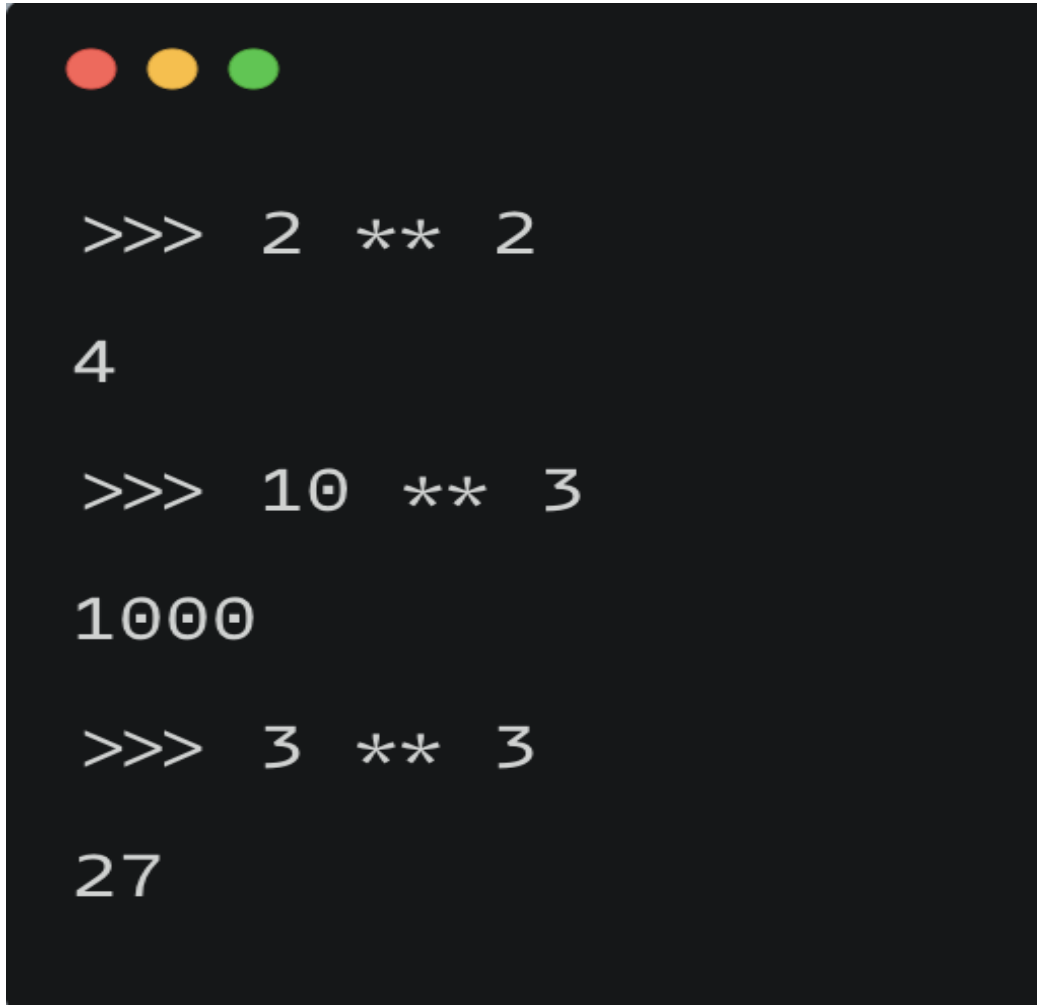
INTEGERS

- You can add, subtract, multiply and divide python integers

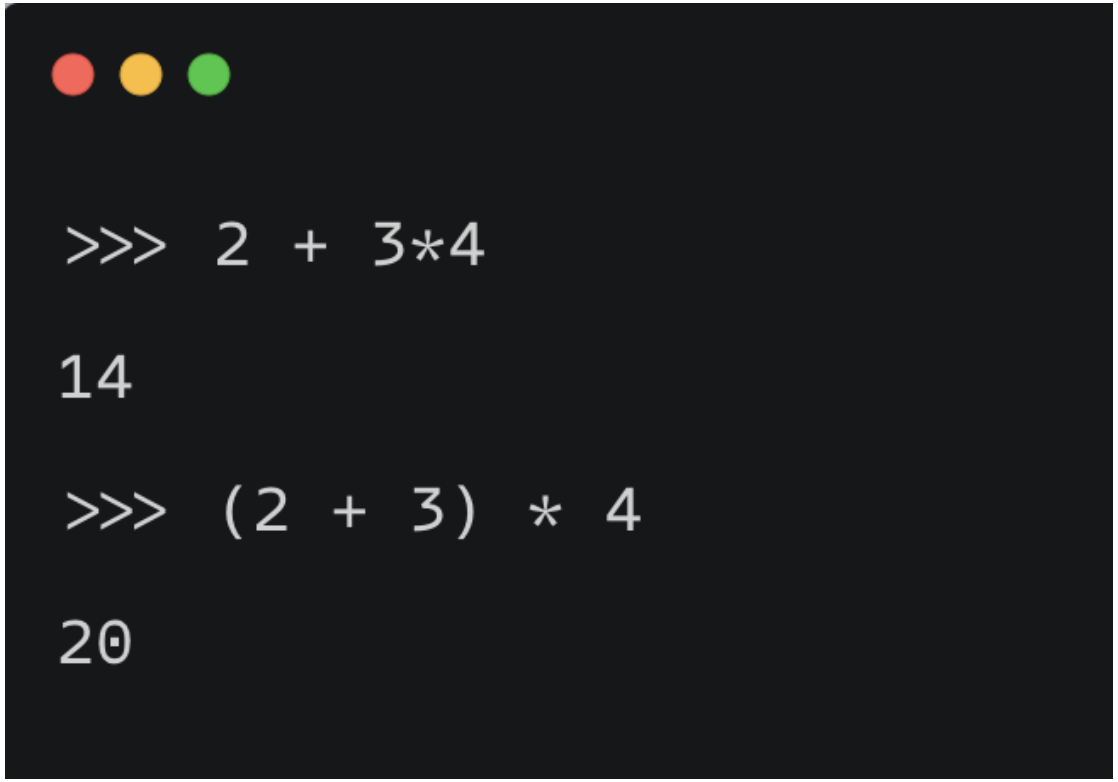



>>> x = 3
>>> y = 2
>>> print(x + y)
5
>>> print(x - y)
1
>>> print(x * y)
6
>>> print(x / y)
1.5

Python uses two multiplication symbols to represent exponents.



MULTIPLE

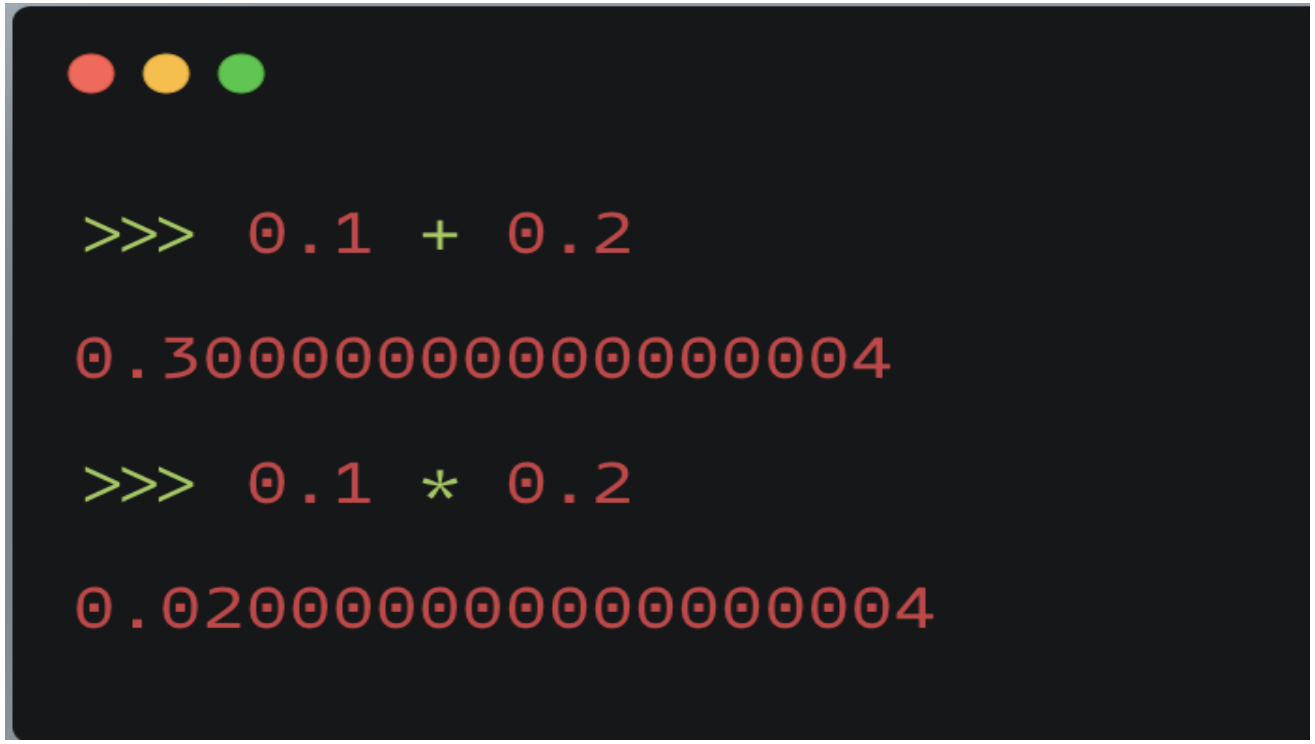


- The spacing in these examples has no effect on how Python evaluates the expressions
- Clarity
- More about precedence of operators in a future video

FLOATS

- Python calls any number with a decimal point a *float*.
- This term is used in most programming languages, and it refers to the fact that a decimal point can appear at any position in a number.
- Example: 1.2, 12.35


>>> 0.1 + 0.1
0.2
>>> 0.2 + 0.2
0.4
>>> 2 * 0.1
0.2
>>> 2 * 0.2
0.4

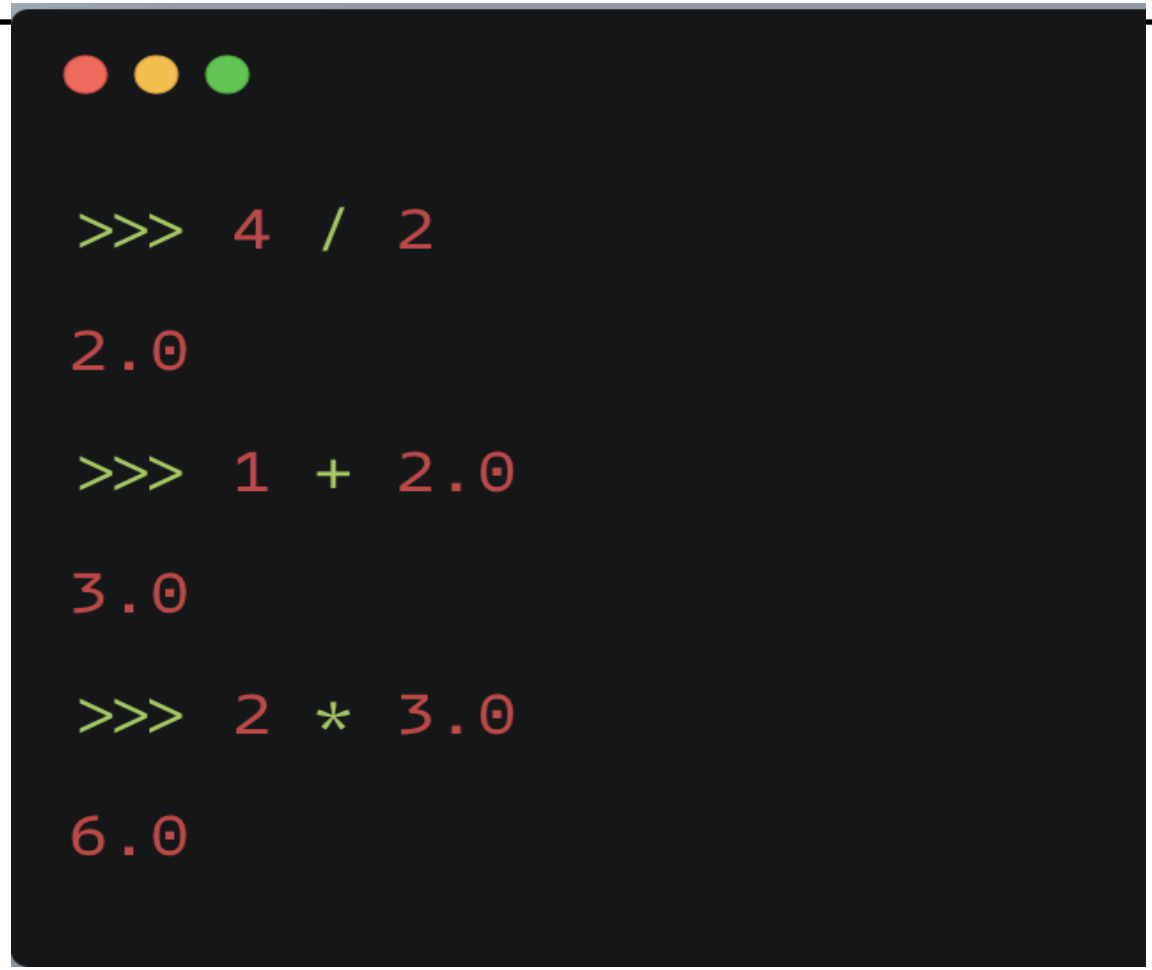


This happens in all languages and is of little concern. Python tries to find a way to represent the result as precisely as possible, which is sometimes difficult given how computers have to represent numbers internally.

USE ROUND FUNCTION

```
>>> x = 0.1 + 0.2
>>> x
0.30000000000000004
>>> print(round(x))
0
>>> print(round(x,1))
0.3
```

INTEGERS AND FLOATS

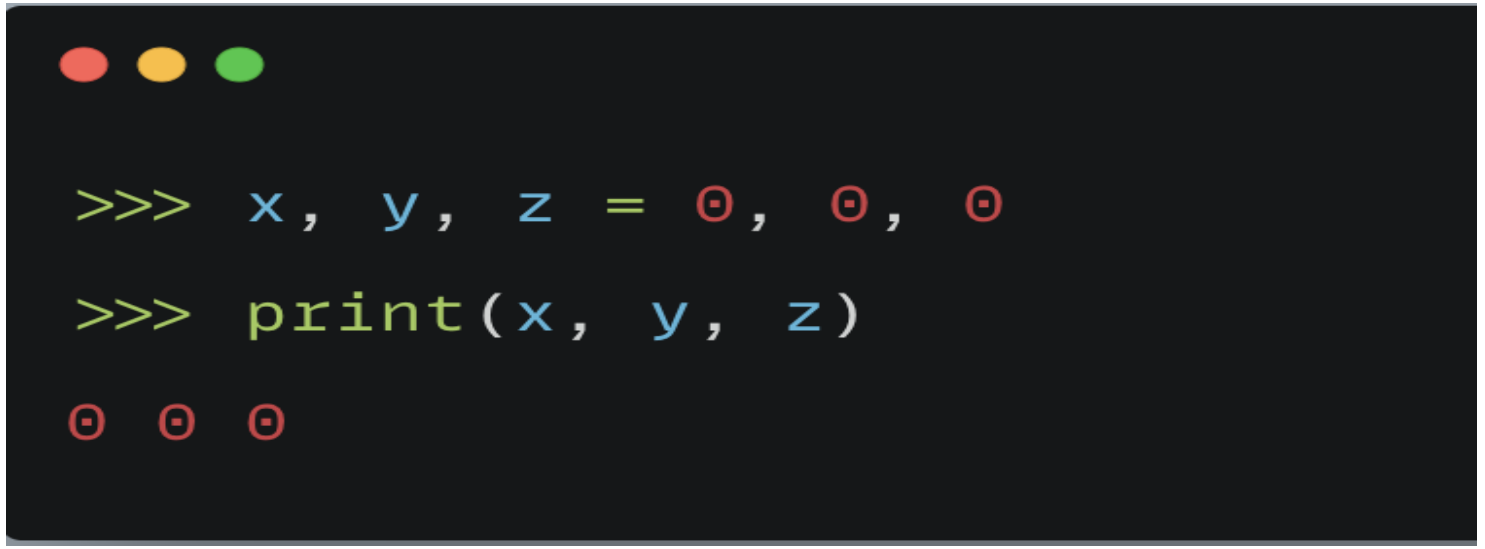


DATA TYPE CONVERSION



MULTIPLE ASSIGNMENTS

- You can assign values to more than one variable using just a single line.
- This can help shorten your programs and make them easier to read.
- You'll use this technique most often when initializing a set of numbers.



CONSTANTS IN PYTHON

- A *constant* is like a variable whose value stays the same throughout the life of a program.
- Python doesn't have built-in constant types, but Python programmers use all capital letters to indicate a variable should be treated as a constant and never be changed.

```
>>> MAX_SIZE = 50
>>> print(MAX_SIZE)
50
>>> MAX_SIZE = 20
>>> print(MAX_SIZE)
20
```

LISTS IN PYTHON

STORING MULTIPLE NAMES ?

- 5 names, 10 names …. 100 names
- Storing it in strings doesn’t scale well
- Storing multiple numbers
- Code clarity
- Slow access

INTRODUCING LISTS

- A list is a collection of items in a particular order
- List of
  - Names
  - Places
  - list of digits
  - list of colors
- Good to name your list names as plurals – names, places etc

LISTS IN PYTHON

- Use square brackets ([]) to represent a list
- Individual elements are separated by a comma
- Printing a list

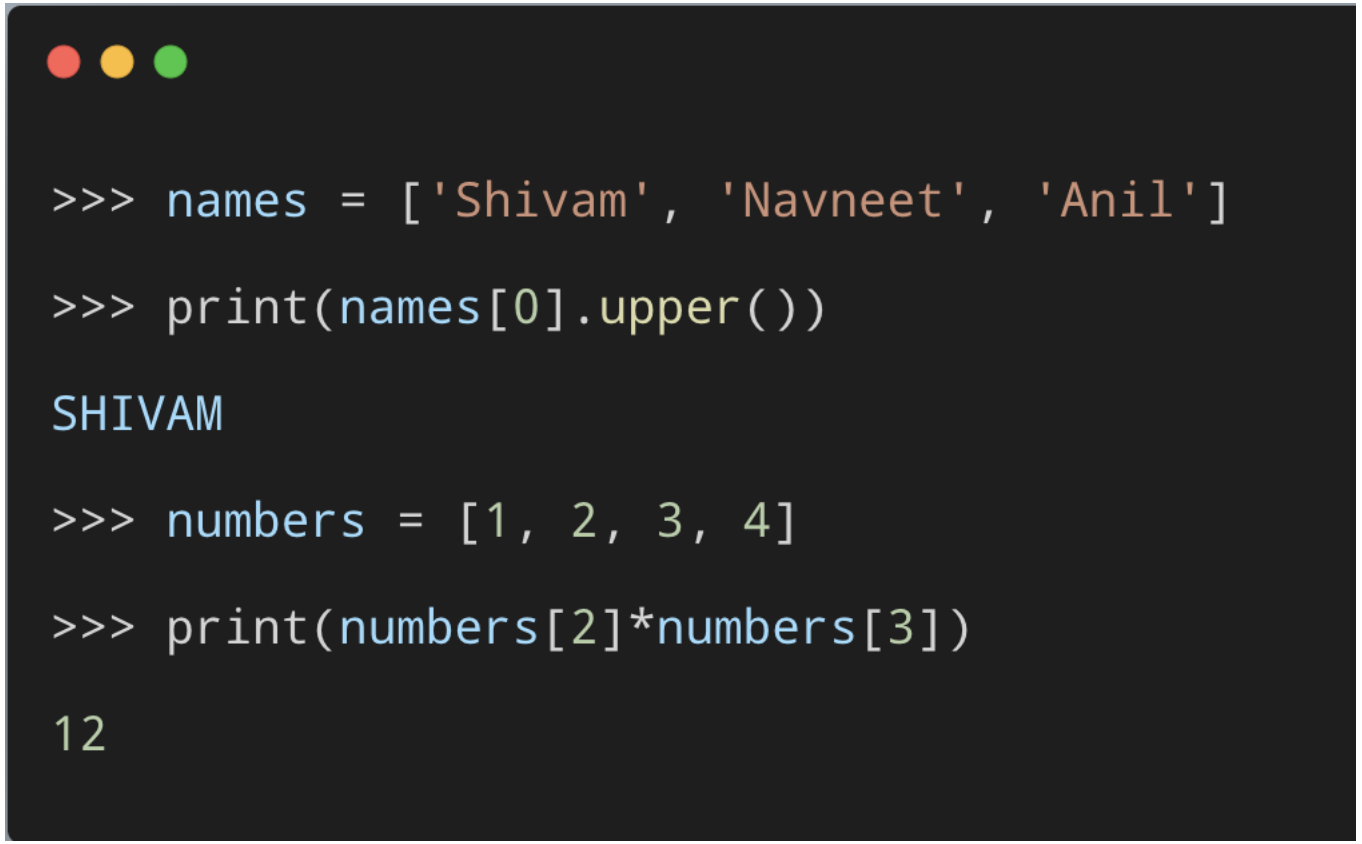
```
>>> names = ['Shivam', 'Navneet', 'Anil']
>>> print(names)
['Shivam', 'Navneet', 'Anil']
>>> names
['Shivam', 'Navneet', 'Anil']
```

ACCESSING ELEMENTS IN A LIST

- Lists are ordered
- Access an element by its position or index
- In Python and most languages, list numbering starts from 0th position
- This is very important

```
>>> names = ['Shivam', 'Navneet', 'Anil']
>>> print(names[0])
Shivam
>>> print(names[1], names[2])
Navneet Anil
```

USE STRING METHODS ON THESE ELEMENTS

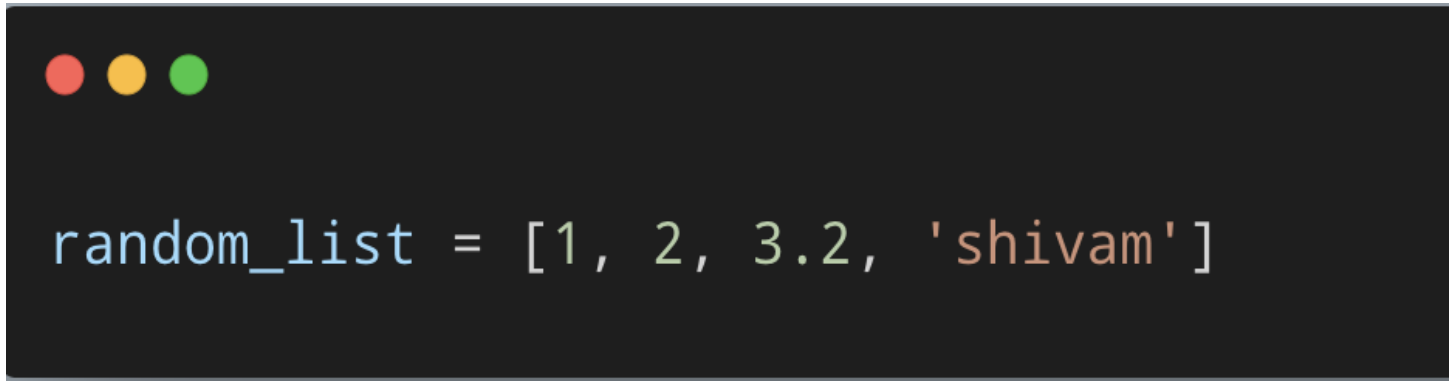


```
>>> names = ['Shivam', 'Navneet', 'Anil']
>>> print(names[0].upper())
SHIVAM
>>> numbers = [1, 2, 3, 4]
>>> print(numbers[2]*numbers[3])
12
```

REMEMBER THIS IMPORTANT THING

- ✓ Each index in a list contains a data type – integers, float, strings etc
- ✓ Instead to using multiple variable names, you are using a single variable name to store all these
- ✓ All functions/operations on a data type applies here to elements

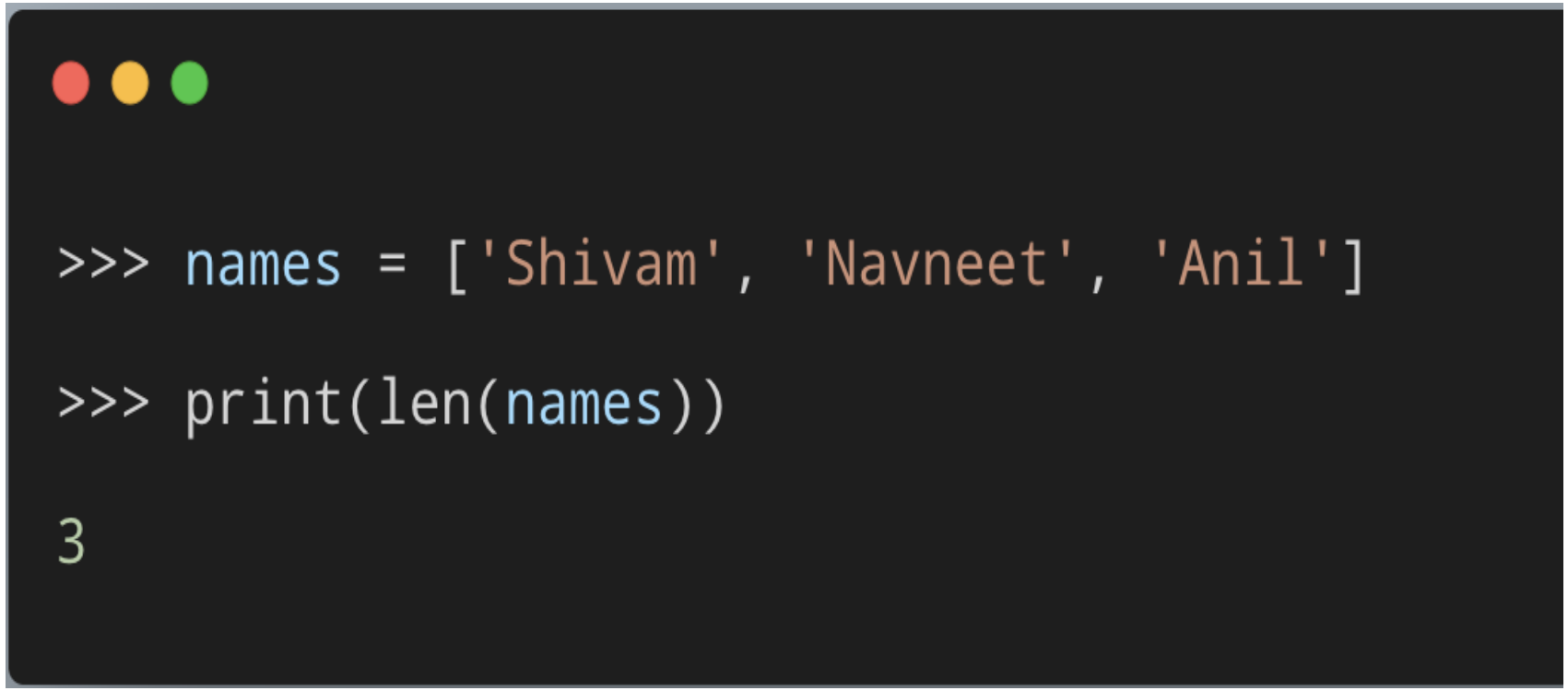
CAN WE STORE DIFFERENT DATA TYPES IN A LIST ?



```
random_list = [1, 2, 3.2, 'shivam']
```

But mostly we will store same type of data

LENGTH OF THE LIST USING LEN() FUNCTION



ACCESSING LAST ELEMENTS IN A LIST

```
>>> names = ['Shivam', 'Navneet', 'Anil']
>>> last_element_index = len(names) - 1
>>> print(names[last_element_index])
Anil
```

NEGATIVE INDEXING IN PYTHON



```
>>> names = ['Shivam', 'Navneet', 'Anil']
>>> last_element_index = len(names) - 1
>>> print(names[last_element_index])
Anil
```

THIS WILL BE FASTER THAN THE LAST METHOD

STRINGS VS LISTS IN PYTHON



MODIFYING AN ELEMENT - LIST VS STRING

```
>>> names = ['Shivam', 'Navneet', 'Anil']
>>> names[0] = 'Rohit'
>>> names
['Rohit', 'Navneet', 'Anil']
>>> names[0]
'Rohit'
```

```
>>> message = 'hello world'
>>> message[0] = 'H'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
```

MUTABLE VS IMMUTABLE DATA TYPES

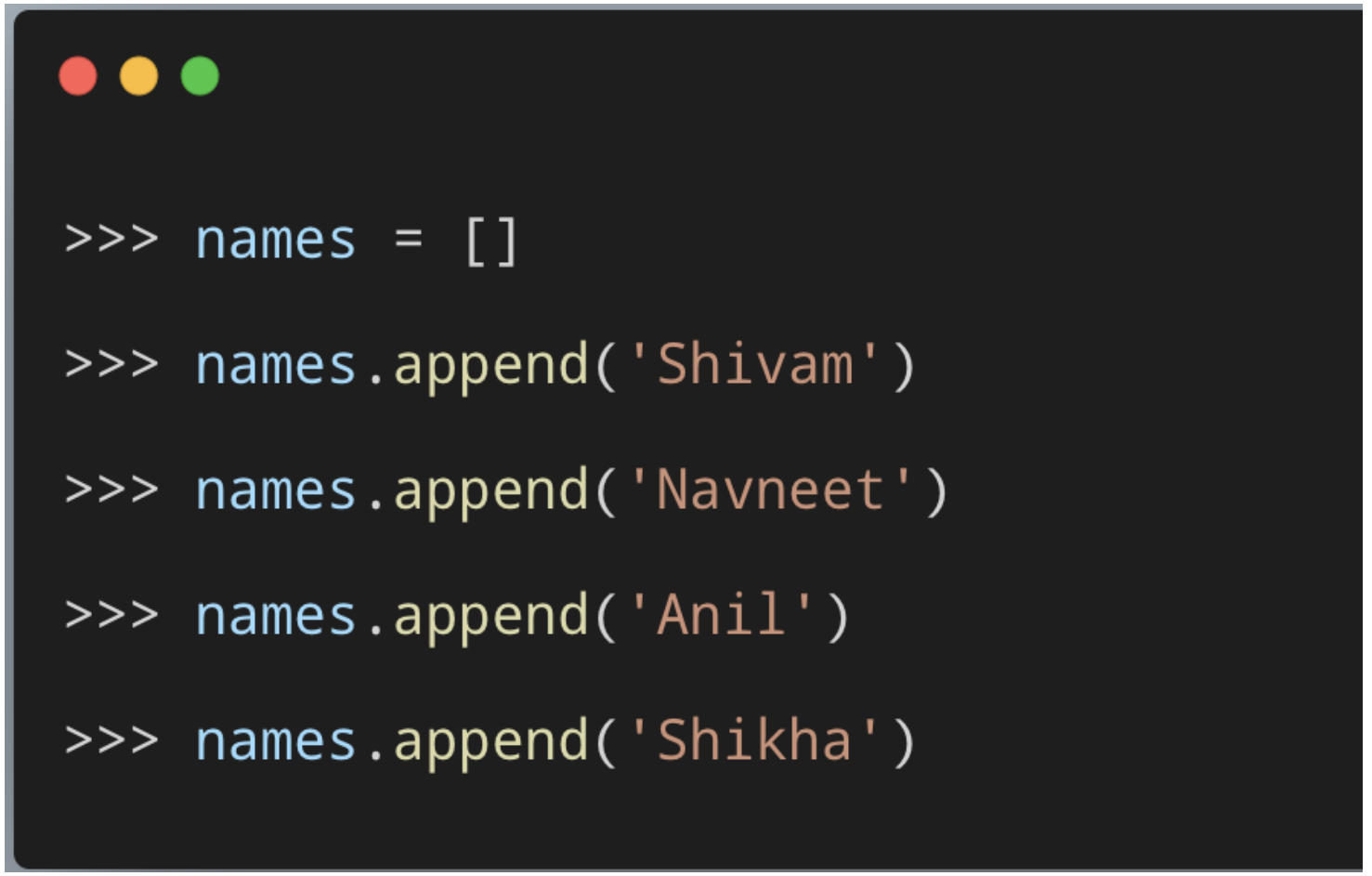
- Immutable data type
  - The value/data cannot be changed
  - Example: strings, integers, floats etc
- Mutable data type
  - The value/data can be changed
  - Example – lists
- More about this in a separate video

```
>>> message = 'hello world'
>>> message.upper()
'HELLO WORLD'
>>> print(message)
hello world
>>> message = message.upper()
>>> print(message)
HELLO WORLD
```

INSERTING ELEMENTS AT THE END OF A LIST

```
>>> names = ['Shivam', 'Navneet', 'Anil']
>>> names = ['Shivam', 'Navneet', 'Anil', 'Shikha']
```

```
>>> names = ['Shivam', 'Navneet', 'Anil']
>>> names.append('Shikha')
>>> names
['Shivam', 'Navneet', 'Anil', 'Shikha']
```



- This is very common
- As data is mostly known after the program starts running

INSERTING ELEMENTS AT ANY POSITION

```
>>> names = ['Shivam', 'Navneet', 'Anil']

>>> names.insert(0, 'Shikha')

>>> names

['Shikha', 'Shivam', 'Navneet', 'Anil']
```

```
>>> names = ['Shivam', 'Navneet', 'Anil']
>>> names.insert(3, 'Shikha')
>>> names
['Shivam', 'Navneet', 'Anil', 'Shikha']
```

```
>>> names = ['Shivam', 'Navneet', 'Anil']
>>> names.insert(-1, 'Shikha')
>>> names
['Shivam', 'Navneet', 'Shikha', 'Anil']
```

REMOVING ELEMENTS FROM A LIST

REMOVING USING DEL STATEMENTS

```
>>> names = ['Shivam', 'Navneet', 'Anil', 'Shikha']
>>> del names[-1]
>>> names
['Shivam', 'Navneet', 'Anil']
```

REMOVING USING POP METHOD

The pop() method removes the last item in a list, but it lets you work with that item after removing it

```
>>> names = ['Shivam', 'Navneet', 'Anil', 'Shikha']
>>> popped_name = names.pop()
>>> print(popped_name)
Shikha
>>> print(names)
['Shivam', 'Navneet', 'Anil']
```

POPPING ITEMS FROM ANY POSITION IN THE LIST

```
>>> names = ['Shivam', 'Navneet', 'Anil', 'Shikha']
>>> popped_name = names.pop(0)
>>> print(names)
['Navneet', 'Anil', 'Shikha']
>>> print(names)
['Navneet', 'Anil', 'Shikha']
```

WHEN TO USE DEL VS POP ?

If you're unsure whether to use the `del` statement or the `pop()` method, here's a simple way to decide: when you want to delete an item from a list and not use that item in any way, use the `del` statement; if you want to use an item as you remove it, use the `pop()` method.

REMOVING AN ELEMENT BY VALUE

```
>>> names = ['Shivam', 'Navneet', 'Anil', 'Shikha']
>>> name_to_remove = 'Anil'
>>> names.remove(name_to_remove)
>>> names
['Shivam', 'Navneet', 'Shikha']
```

REMOVING AN ELEMENT NOT IN A LIST

```
>>> names = ['Shivam', 'Navneet', 'Anil', 'Shikha']
>>> name_to_remove = 'Ani'
>>> names.remove(name_to_remove)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: list.remove(x): x not in list
```

CAN DUPLICATES BE REMOVED ?

```
>>> names = ['Shivam', 'Navneet', 'Anil', 'Anil']
>>> name_to_remove = 'Anil'
>>> names.remove(name_to_remove)
>>> names
['Shivam', 'Navneet', 'Anil']
```

- Removes first matching value
- USING REMOVE AGAIN WILL DO

ORGANIZING A LIST

SORTING A LIST

- Putting the elements in a particular order
- Increasing order
- Decreasing order
- Numbers = [2, 1, 3]
- Increasing = [1, 2, 3]
- Decreasing = [3, 2, 1]

SORTING A LIST OF STRINGS

- Alphabetical order
- Names = [‘Shivam’, ’Anil’, ‘Navneet’]
- Increasing alphabetical order = [‘Anil’, ‘Navneet’, ‘Shivam’]
- Decreasing alphabetical order = [‘Shivam’, ‘Navneet’, ‘Anil’]

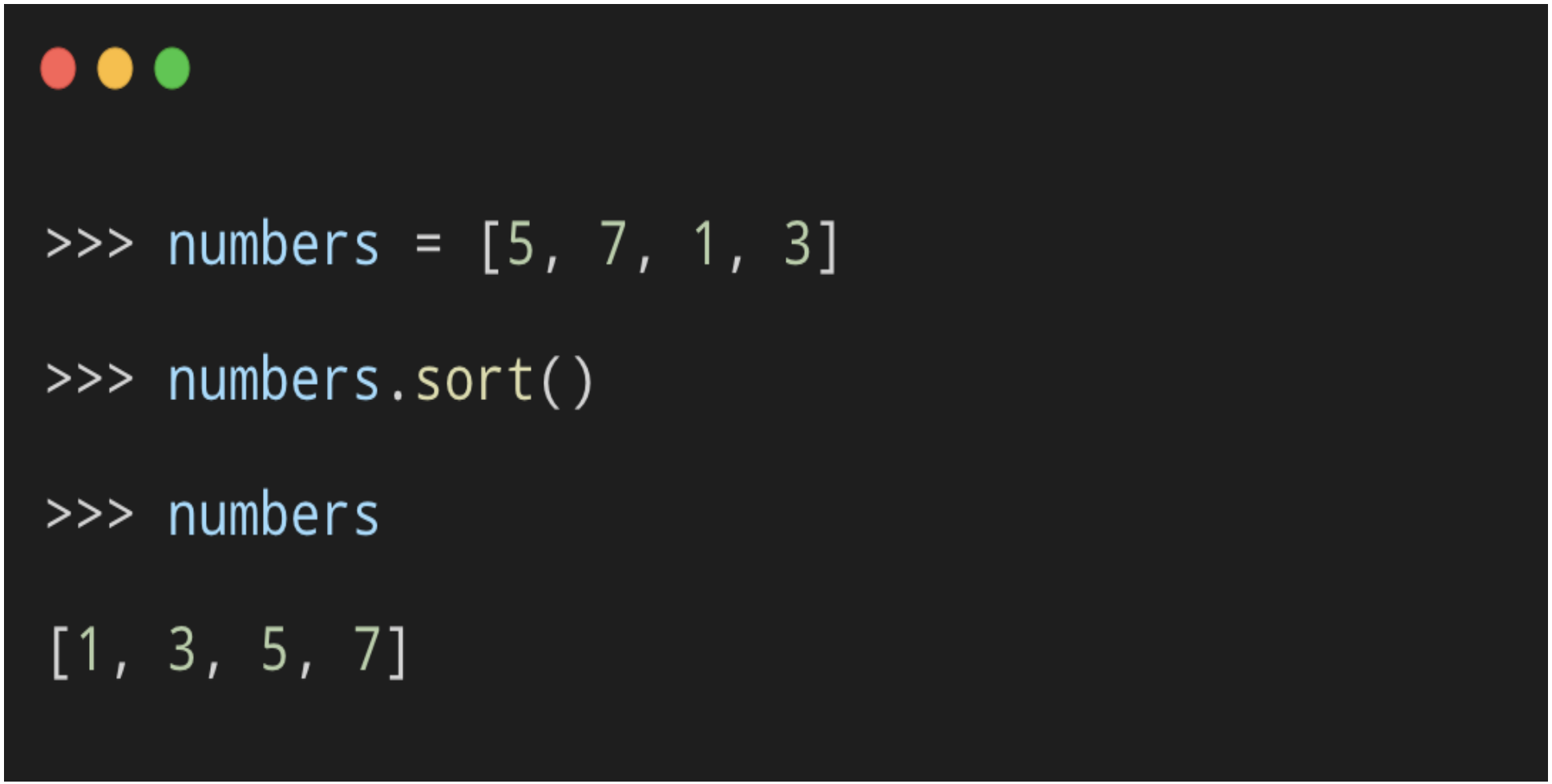
SORTING A LIST PERMANENTLY WITH SORT FUNCTION

```
>>> names = ['Shivam', 'Anil', 'Navneet']
>>> names.sort()
>>> names
['Anil', 'Navneet', 'Shivam']
```

BY DEFAULT, SORT() SHORTES IN INCREASING ORDER

SORTING IN REVERSE ALPHABETIC ORDER

```
>>> names = ['Shivam', 'Anil', 'Navneet']
>>> names.sort(reverse = True)
>>> names
['Shivam', 'Navneet', 'Anil']
```


>>> numbers = [5, 7, 1, 3]
>>> numbers.sort()
>>> numbers
[1, 3, 5, 7]

SORTING A LIST TEMPORARILY WITH SORTED FUNCTION

To maintain the original order of a list but present it in a sorted order, you can use the sorted() function.

The sorted() function lets you display your list in a particular order but doesn't affect the actual order of the list.


>>> names = ['Shivam', 'Anil', 'Navneet']
>>> print(names)
['Shivam', 'Anil', 'Navneet']
>>> print(sorted(names))
['Anil', 'Navneet', 'Shivam']
>>> print(names)
['Shivam', 'Anil', 'Navneet']

PRINTING A LIST IN REVERSE ORDER

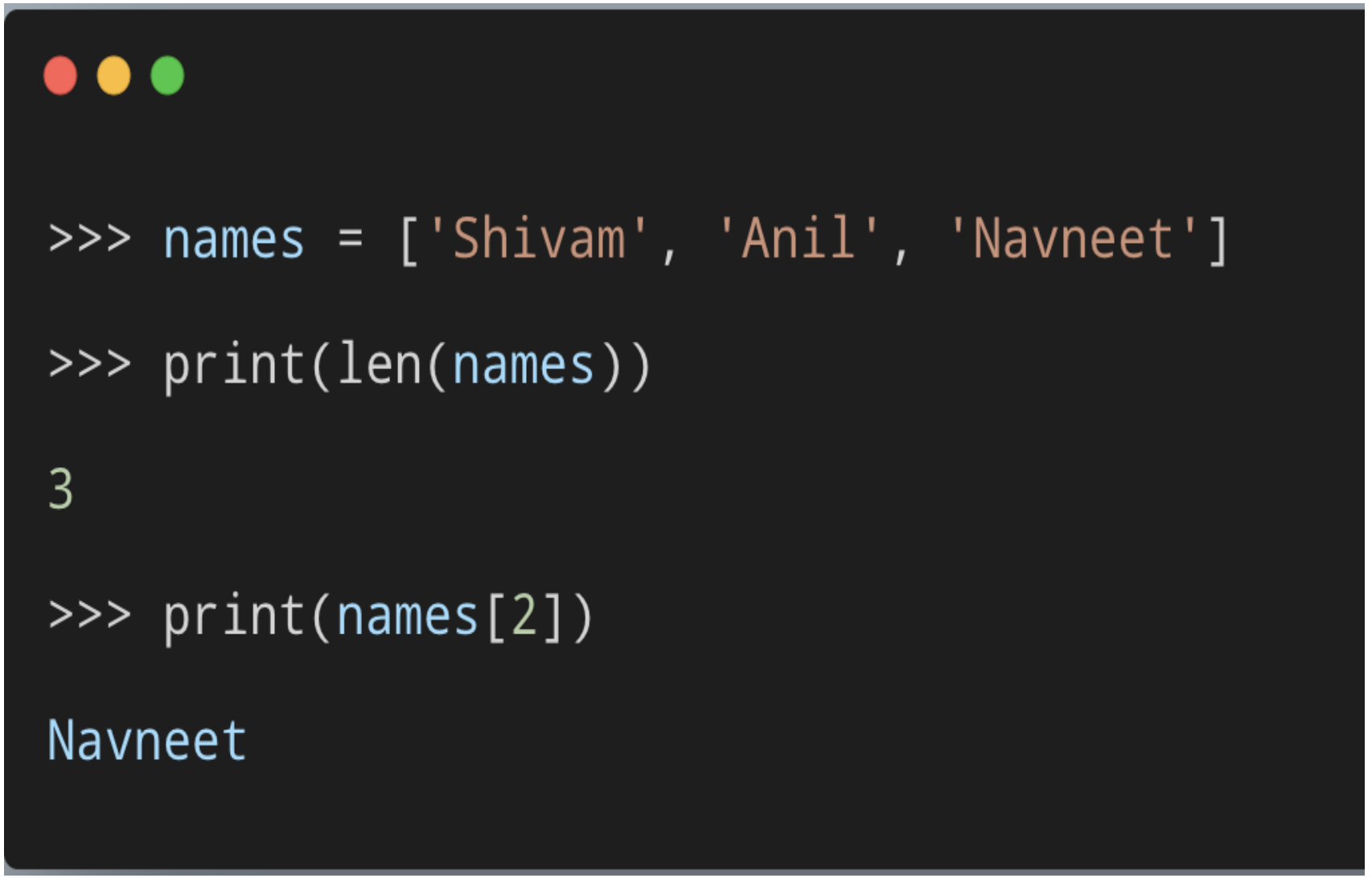
```
>>> names = ['Shivam', 'Anil', 'Navneet']
>>> print(names)
['Shivam', 'Anil', 'Navneet']
>>> names.reverse()
>>> print(names)
['Navneet', 'Anil', 'Shivam']
```

What if you reverse again?

AVOIDING INDEX ERRORS WHEN WORKING WITH LIST

```
>>> names = ['Shivam', 'Anil', 'Navneet']
>>> print(names[3])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
```

REMEMBER THAT INDEXING STARTS AT 0


>>> names = ['Shivam', 'Anil', 'Navneet']
>>> print(len(names))
3
>>> print(names[2])
Navneet

Thank You